UNIVERSITY SCHOLARSHIP ALLOCATION SYSTEM

Project Report — Python Programming

Project purpose	Evaluate applicants against academic, attendance, and income rules
Eligibility	Aggregate marks, subject minimums, attendance threshold, income ceiling
Ranking	Aggregate percentage → attendance percentage → lower family income → applicant name
Implementation	Python 3, console-based program

All thresholds in this report are example policy values and should be replaced with the university's official scholarship rules. Sample output is illustrative, not a captured screenshot.

1. Problem Statement

Universities receive scholarship applications from students with different academic results, attendance records, and family incomes. Manually checking each application and ranking eligible candidates can be time-consuming and inconsistent. This project develops a Python-based system that checks applicants against defined scholarship criteria, records the reasons for ineligibility, ranks eligible applicants, resolves ties using predefined rules, and displays a final merit list.

2. Objectives

• Collect applicant names, subject marks, attendance percentage, and annual family income.

• Calculate total marks and aggregate percentage.

• Check aggregate, subject-wise minimum, attendance, and income requirements.

• Classify each applicant as eligible or ineligible and explain failed criteria.

• Rank eligible applicants using a transparent, deterministic tie-break policy.

• Display the merit list and a summary of ineligible applications.

3. Assumptions

• Example policy: aggregate percentage must be at least 75%.

• Each subject mark must be at least 40 out of 100.

• Attendance must be at least 75%.

• Annual family income must be at or below ■250,000.

• Ranking order: aggregate percentage descending, attendance descending, annual family income ascending, then applicant name alphabetically.

• All marks are whole numbers from 0 to 100; attendance is 0–100%; income is a non-negative amount in rupees.

• The program is a demonstration and does not verify documents or make an official funding decision.

4. Algorithm

1. Read the number of applicants and number of subjects.

2. Input unique applicant names and subject names.

3. For each applicant, read marks for every subject, attendance percentage, and annual family income.

4. Validate all numeric inputs and calculate aggregate percentage as total marks divided by maximum possible marks, multiplied by 100.

5. Check aggregate threshold, each subject minimum, attendance threshold, and income ceiling.

6. If any condition fails, mark the applicant ineligible and store each reason.

7. Sort eligible applicants by aggregate percentage descending, attendance descending, income ascending, and name ascending.

8. Display the ranked merit list and the ineligible list with reasons.

5. Flowchart

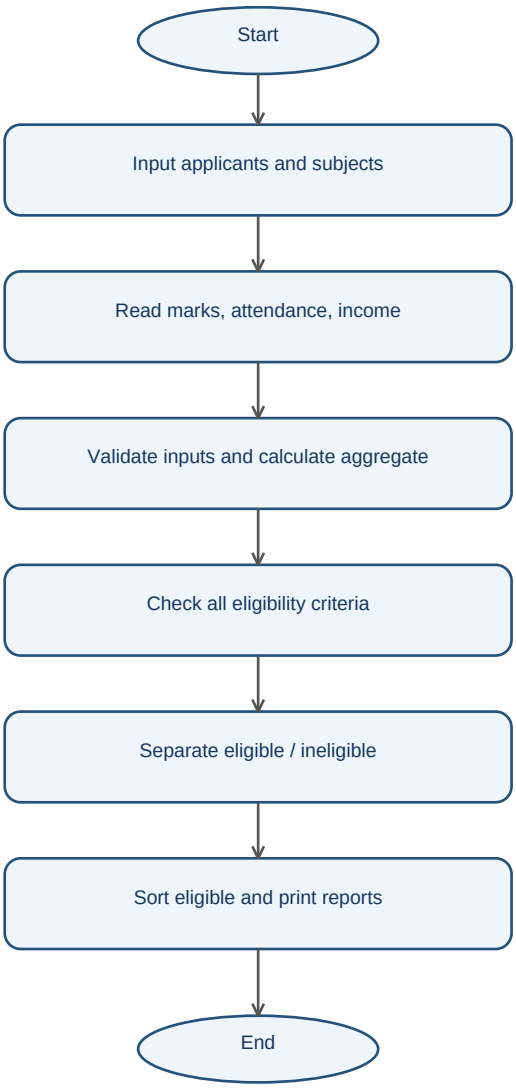

Figure 1. High-level workflow of the scholarship allocation system.

6. Pseudocode

```
START
  INPUT applicant_count, subject_count
  INPUT unique subject names
  FOR each applicant
    INPUT unique applicant name
    INPUT marks for each subject
    VALIDATE each mark is between 0 and 100
    INPUT attendance percentage and annual family income
    aggregate = total_marks / (subject_count * 100) * 100
    reasons = empty list
    IF aggregate < 75: add aggregate failure reason
    IF any subject mark < 40: add subject failure reason(s)
    IF attendance < 75: add attendance failure reason
    IF income > 250000: add income failure reason
    IF reasons is empty: mark eligible
    ELSE: mark ineligible and store reasons
  END FOR

  SORT eligible applicants by:
    aggregate descending,
    attendance descending,
    income ascending,
    name ascending
  DISPLAY ranked eligible list
  DISPLAY ineligible list with reasons
END
```

7. Python Source Code

```
AGGREGATE_MIN = 75.0
SUBJECT_MIN = 40
ATTENDANCE_MIN = 75.0
INCOME_MAX = 250000.0

def read_int(prompt, minimum, maximum=None):
    while True:
        try:
            value = int(input(prompt))
            if value < minimum or (maximum is not None and value > maximum):
                limit = f" and at most {maximum}" if maximum is not None else ""
                print(f"Enter a whole number at least {minimum}{limit}.")
                continue
            return value
        except ValueError:
            print("Invalid input. Enter a whole number.")

def read_float(prompt, minimum, maximum=None):
    while True:
        try:
            value = float(input(prompt))
            if value < minimum or (maximum is not None and value > maximum):
                limit = f" and at most {maximum}" if maximum is not None else ""
                print(f"Enter a number at least {minimum}{limit}.")
                continue
            return value
        except ValueError:
            print("Invalid input. Enter a number.")

def read_unique_names(count, label):
    names = []
    while len(names) < count:
        name = input(f"Enter {label} {len(names) + 1}: ").strip()
        if not name:
            print("Name cannot be empty.")
        elif name.casefold() in [item.casefold() for item in names]:
            print("Names must be unique.")
        else:
            names.append(name)
    return names

def main():
    applicant_count = read_int("Number of applicants: ", 1, 200)
    subject_count = read_int("Number of subjects: ", 1, 30)
    subjects = read_unique_names(subject_count, "subject")
    eligible, ineligible = [], []

    for i in range(applicant_count):
        print(f"\\nApplicant {i + 1}")
        # Applicant names must be unique across the whole list.
        name = input("Applicant name: ").strip()
        while not name or any(a["name"].casefold() == name.casefold()
                              for a in eligible + ineligible):
            print("Name must be non-empty and unique.")
            name = input("Applicant name: ").strip()

        marks = []
```

```
            for subject in subjects:
                marks.append(read_int(f"Marks in {subject} (0-100): ", 0, 100))
            attendance = read_float("Attendance percentage (0-100): ", 0, 100)
            income = read_float("Annual family income in rupees: ", 0)

            aggregate = sum(marks) / (len(subjects) * 100) * 100
            reasons = []
            if aggregate < AGGREGATE_MIN:
                reasons.append(f"aggregate below {AGGREGATE_MIN:.0f}%")
            for subject, mark in zip(subjects, marks):
                if mark < SUBJECT_MIN:
                    reasons.append(f"{subject} mark below {SUBJECT_MIN}")
            if attendance < ATTENDANCE_MIN:
                reasons.append(f"attendance below {ATTENDANCE_MIN:.0f}%")
            if income > INCOME_MAX:
                reasons.append(f"income exceeds Rs. {INCOME_MAX:.0f}")

            record = {"name": name, "aggregate": aggregate,
                      "attendance": attendance, "income": income}
            if reasons:
                record["reasons"] = reasons
                ineligible.append(record)
            else:
                eligible.append(record)

        # Tie-break policy: aggregate desc, attendance desc, income asc, name asc.
        eligible.sort(key=lambda a: (-a["aggregate"], -a["attendance"],
                                     a["income"], a["name"].casefold()))

        print("\\nELIGIBLE SCHOLARSHIP MERIT LIST")
        print("-" * 62)
        print(f"{'Rank':<6}{'Applicant':<20}{'Aggregate':>11}"
              f"{'Attendance':>12}{'Income (Rs)':>13}")
        print("-" * 62)
        if not eligible:
            print("No applicants meet all eligibility criteria.")
        for rank, a in enumerate(eligible, 1):
            print(f"{rank:<6}{a['name']:<20}{a['aggregate']:>10.2f}%"
                  f"{a['attendance']:>11.2f}%{a['income']:>13.2f}")

        print("\\nINELIGIBLE APPLICANTS")
        if not ineligible:
            print("None")
        for a in ineligible:
            print(f"- {a['name']}: " + "; ".join(a["reasons"]))

    if __name__ == "__main__":
        main()
```

8. Test Cases

Test	Scenario	Expected result
TC-01	Aggregate 80%, all subjects ≥40, attendance 90%, income ■200,000	Eligible
TC-02	Aggregate 70%, other criteria pass	Ineligible: aggregate below 75%
TC-03	Aggregate 80%, one subject mark 35	Ineligible: subject minimum not met
TC-04	Aggregate 80%, attendance 70%, income within limit	Ineligible: attendance below 75%
TC-05	Aggregate 80%, attendance 80%, income ■300,000	Ineligible: income exceeds ■250,000
TC-06	Two eligible applicants with same aggregate; attendance 90% vs 85%	Higher attendance ranks first
TC-07	Same aggregate and attendance; income ■150,000 vs ■200,000	Lower income ranks first
TC-08	All applicants fail at least one condition	Merit list reports no eligible applicants; reasons are listed

9. Sample Output / Console Screenshot

Example policy values are used: aggregate ≥75%, every subject mark ≥40, attendance ≥75%, and annual family income ≤■250,000. Sample applicants: Asha (Maths 85, English 80; attendance 90%; income ■180,000), Ravi (75, 78; 80%; ■200,000), Meena (90, 92; 70%; ■150,000).

```
ELIGIBLE SCHOLARSHIP MERIT LIST
--------------------------------------------------------------
Rank  Applicant            Aggregate  Attendance  Income (Rs)
```

```
------------------------------------------------------------
1     Asha                    82.50%      90.00%    180000.00
2     Ravi                    76.50%      80.00%    200000.00

INELIGIBLE APPLICANTS
- Meena: attendance below 75%
```

This is an illustrative console transcript, not an actual screenshot captured from a running program. For real screenshots, run the program in IDLE or VS Code, enter sample data, and capture the code/output window using Win + Shift + S on Windows.

10. Results and Discussion

The system applies each scholarship condition independently and records all failed criteria, so an applicant can see why they were not eligible. Eligible applicants are sorted with a documented tie-break sequence. Because thresholds are constants near the top of the source code, the policy can be adjusted to match the official university rules.

11. Conclusion

The University Scholarship Allocation System demonstrates how Python can automate eligibility checking, aggregate calculation, validation, sorting, and merit-list generation. It improves consistency and transparency compared with manual review. It should be treated as a decision-support tool: the university must verify applicant documents, confirm official policy values, and make final scholarship decisions through its authorized process.

12. How to Run

• Save the code in a file named scholarship_allocation.py.

• Run it using Python 3: python scholarship_allocation.py.

• Enter subject names, applicant details, marks, attendance, and annual income.

• Review the merit list and the reasons shown for ineligible applicants.